KR MANGALAM UNIVERSITY

SECURE CODING AND VULNERABILTIES LAB

(ENSP 351)

NAME – YASH VASHISTH

B.TECH CSE AI/ML SEC-C

ROLL NUMBER - 2301730149

Assignment 2

Compiler Hardening Flags and UBSan/ASan Diagnostics

1. Aim

To compile the type-conversion program under different compiler configurations, compare compile-time warnings and runtime sanitizer diagnostics, and identify which defects each configuration detects or misses.

2. Objectives

- Compile the diagnostic program without additional warning or sanitizer flags.
- Compile using -Wall -Wextra.
- Compile using -fsanitize=undefined (UBSan).
- Compile using -fsanitize=address (ASan).
- Capture compiler output and runtime diagnostics.
- Compare which defects are detected and which are missed.
- Explain how compiler hardening contributes to secure C++ development.

3. Methodology

A deliberately defective C++ program was prepared using the same conversion concepts as Assignment 1. The program contains narrowing conversion, signed-to-unsigned conversion, floating-point truncation, signed integer overflow, array out-of-bounds access, and string-to-int conversion. It is then compiled under four configurations so that static compiler diagnostics can be compared with runtime sanitizer diagnostics.

4. Diagnostic Implementation

```
#include <iostream>
#include <string>
#include <limits>
using namespace std;

int main() {
    // 1. int -> short     int
largeInt = 40000;     short
shortValue = largeInt;

    cout << "INT -> SHORT: "
<< shortValue << endl;

    // 2. signed -> unsigned
int negativeValue = -10;
    unsigned int unsignedValue = negativeValue;

    cout << "SIGNED -> UNSIGNED: "
<< unsignedValue << endl;

    // 3. float -> int     float
decimalValue = 25.75f;     int
integerValue = decimalValue;

    cout << "FLOAT -> INT: "
<< integerValue << endl;

    // 4. Signed integer overflow     int
maxInt = numeric_limits<int>::max();

    cout << "INTEGER OVERFLOW\n";

int overflowValue = maxInt + 1;

cout << overflowValue << endl;

// 5. Array out-of-bounds     cout

<< "ARRAY OUT-OF-BOUNDS\n";     int

values[3] = {10, 20, 30};


    // Intentional invalid access
cout << values[5] << endl;

    // 6. String -> int
string input = "9999999999";
```

```
    try {         int converted =
stoi(input);          cout << "STRING
-> INT: "               << converted
<< endl;
    }
    catch (const exception& e) {
        cout << "STRING CONVERSION ERROR: "
             << e.what() << endl;
    }

    return 0;
}
```

5. Compilation Commands

Configuration 1 — No flags

```
g++ conversion_diagnostics.cpp -o conversion_none
```

Configuration 2 — -Wall -Wextra

```
g++ -Wall -Wextra conversion_diagnostics.cpp -o conversion_warnings
```

Configuration 3 — UndefinedBehaviorSanitizer

```
g++ -Wall -Wextra -fsanitize=undefined -g conversion_diagnostics.cpp -o conversion_ubsan
```

Configuration 4 — AddressSanitizer

```
g++ -Wall -Wextra -fsanitize=address -g conversion_diagnostics.cpp -o conversion_asan
```

Run the executables after compilation. On Linux/WSL, for example:

```
./conversion_none
./conversion_warnings
./conversion_ubsan
./conversion_asan
```

6. Representative Compiler/Runtime Diagnostics

The following are representative forms of diagnostics. Exact wording depends on the compiler, compiler version, operating system, architecture, and flags. The final lab submission should replace these excerpts with the actual outputs captured during the experiment.

6.1 -Wall -Wextra

```
Possible compiler warning with stronger conversion diagnostics:
warning: conversion from 'int' to 'short int' may change value warning:
conversion from 'int' to 'unsigned int' may change the sign
```

Important limitation: -Wall -Wextra does not mean 'all possible warnings'. In particular, GCC may require conversion-specific options such as -Wconversion and -Wsign-conversion to diagnose some implicit numeric conversions. Therefore the exact warning output must be recorded from the actual compiler used.

6.2 UBSan

```
Example runtime diagnostic:
runtime error: signed integer overflow: 2147483647
+ 1 cannot be represented in type 'int'
```

This corresponds to the statement maxInt + 1. Signed integer overflow is undefined behavior, so UBSan can instrument the program to diagnose it when that operation executes.

6.3 ASan

```
Example runtime diagnostic:
ERROR: AddressSanitizer: stack-buffer-overflow
READ of size 4 in main
...
values[5]
...
SUMMARY: AddressSanitizer: stack-buffer-overflow
```

This corresponds to accessing values[5] when values contains only three elements, indices 0 through 2.

7. Side-by-Side Diagnostic Comparison

Defect	No flags	-Wall -Wextra	UBSan	ASan
int → short	Usually missed	May warn; stronger conversion warnings may be needed	Not main purpose	Not main purpose
signed → unsigned	Usually missed	May warn; sign-conversion warnings may be needed	Not main purpose	Not main purpose
float → int	Usually missed	May not warn with these flags alone	Not main purpose	Not main purpose
signed integer overflow	Runtime defect	Not necessarily diagnosed at runtime	Detected	Not main purpose
array out-of-bounds	Usually missed	Runtime-dependent	Not main purpose	Detected
string out of int range	Handled by exception	Same application behavior	Not a string-validation tool	Not its purpose

8. Explanation of Hardening Flags

-Wall: enables a broad group of commonly useful compiler warnings. It improves static visibility of suspicious source constructs but does not literally enable every GCC warning.

-Wextra: enables additional warnings beyond the normal -Wall set.

-fsanitize=undefined: adds runtime instrumentation for many forms of undefined behavior. In this experiment, it is intended to identify the signed integer overflow.

-fsanitize=address: adds runtime instrumentation for invalid memory accesses. In this experiment, it is intended to identify the out-of-bounds array access.

-g: includes debugging information, which helps the sanitizer report source locations and stack information.

9. Security Analysis

- Compiler warnings provide early feedback before the program is executed.
- UBSan and ASan expose runtime defects that static warnings may not reliably detect.
- Sanitizers are complementary: UBSan focuses on selected undefined behaviors, while ASan focuses onmemory-access errors.
- Range validation remains necessary because sanitizers do not replace correct application logic.
- Testing must exercise the defective code path; a sanitizer cannot report a bug that execution neverreaches.

10. Result

The experiment demonstrates that different compiler configurations provide different diagnostic coverage. The normal build provides the baseline, -Wall -Wextra improves compile-time warnings, UBSan can expose selected undefined behavior such as signed integer overflow, and ASan can expose invalid memory accesses such as an array out-of-bounds read.

11. Conclusion

Compiler acceptance does not imply that C++ code is safe. Secure development benefits from layered checking: compile-time warnings, explicit application-level validation, and runtime sanitizer testing. No single configuration detects every defect. In particular, -Wall -Wextra should not be treated as an exhaustive numeric-conversion checker, and UBSan/ASan should not be treated as substitutes for safe code design.